

(WiP paper) Navigator: A Microcontroller-based Assistive Smartglass for Guided Navigation

Saket Nerurkar,¹ Vyomesh Bhatt,¹ Surjya Ghosh,^{1,2} Sougata Sen^{1,2}

¹ Dept. of Computer Science and Information Systems, BITS Pilani Goa Campus, India.

² APPCAIR, BITS Pilani Goa Campus, India.

ABSTRACT

Automating the assisted navigation process has been a long-standing research challenge. Although it is possible to develop navigation systems using GPS or RF-based techniques, such systems cannot capture specific context in a scene, and fail in real-time scene-based decision making. Over the years, researchers have proposed various context-aware approaches for assisted navigation, including using wearable cameras and other sensors for scene understanding, and using servers for real-time processing. Transferring data out of the device, however, raises energy, latency, and privacy concerns. With recent technological advances, it is now feasible to execute deep learning models on low-cost, low-powered, tiny embedded microcontroller devices. In this paper, we present *Navigator*, a low-power, low-cost, microcontroller-based smartglass with an embedded camera and time of flight sensor. Navigator can perform computation locally and provide real-time navigational guidance to users. Via a user study, we demonstrate that Navigator can accurately (with over 95% accuracy) enable navigating in indoor spaces while reducing power draw. Overall, such a system, if realized, can enable real-time assistance for individuals with visual impairments, without raising latency and privacy concerns.

CCS CONCEPTS

• **Hardware**; • **Human-centered computing** → **Ubiquitous and mobile computing**; Empirical studies in accessibility;

KEYWORDS

Blind and Visually Impaired, Navigation, Microcontroller-based Smartglass, TensorFlow Lite Micro, Time of Flight Sensor, Camera Sensor.

1 INTRODUCTION

According to a 2024 report, 295 million people suffer from moderate to severe vision impairment, of which 43.3 million have complete blindness [21]. Over the years, researchers have developed several technological solutions that help Blind and Visually Impaired (BVI) individuals navigate through spaces. These solutions primarily focus on collecting sensor data (e.g., ultrasonic sensors, wearable cameras) to detect contextual information around the individual, processing the sensor data (either locally or on a more powerful server), and providing feedback to the individual in a human-understandable way (e.g., voice or haptic feedback). While GPS-based systems can enable outdoor navigation and Wi-Fi (and other similar technologies)-based systems can enable indoor navigation, these systems fail in detecting specific details in a scene, e.g., whether a branch of a tree is protruding out, or whether the arrow on the wall indicates if the user should use the front door to enter the hall. They also fail to capture dynamism in the environment, such as introducing

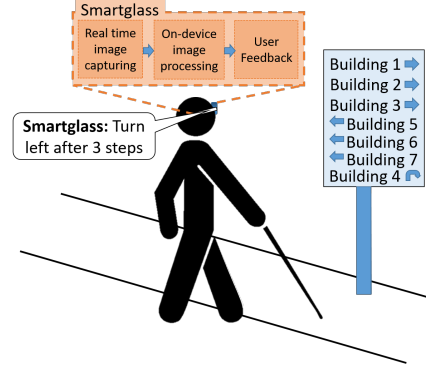


Figure 1: Navigator is a smartglass-based system that is designed to enable people with visual impairment to navigate without assistance. Navigator’s microcontroller runs TinyML on sensor data in real-time to make necessary decisions.

new objects, changing arrows assisting in navigational purposes, or restructuring walking paths. Ranging sensors such as ultrasonic and Time of Flight (ToF) sensors have shown promise in detecting distance to obstacles. Orthogonally, wearable cameras have shown promise in detecting details in scenes. Ranging sensors working in tandem with wearable cameras with monocular vision thus provide tremendous opportunities in navigation assistance. However, a user might have privacy concerns with wearable cameras when images leave the device [8]. Thus, needed are wearable cameras to process data locally and determine specific details in the scene.

In this paper, we seek to answer the question: “Can we develop a low-cost, low-power, privacy-preserving solution to help BVI individuals obtain specific details in a scene during navigation?”. To answer this question, we developed a wearable smartglass system, *Navigator*. Navigator is built around a low-cost microcontroller embedded with a low-power RGB camera and ToF sensors. Navigator uses TensorFlow Lite micro to execute the model in the microcontroller [5]. This approach provides two benefits: (a) it enables the development of a low-latency feedback system, and (b) as the images are not transferred out of the device, it preserves the user’s privacy. Figure 1 presents a high-level overview of Navigator. While Navigator can work in indoor and outdoor environments, in this paper, we experiment with Navigator in indoor environments. We primarily focus on the *real-time signal-processing* aspect of Navigator.

The availability of libraries such as TensorFlow enables seamless development of deep-learning applications; TFLite allows converting these codes for deployment on mobile devices, while tools in TFLite micro enable porting of models and codes onto microcontrollers [12]. Several microcontrollers are supported by TFLite micro [5], including the ESP32-S3 microcontroller from Espressif Systems [1], which we have used in developing Navigator. These microcontrollers have

limited memory and cannot store large deep learning models. Indeed, there are several code-level precautions that we have taken for Navigator to ensure that the complete microcontroller memory is used (details in Section 3), while ensuring that the system is usable for navigational purposes (details in Section 4.2). Specifically, the Navigator system currently detects distance to an obstacle using the ToF sensor, and arrows (a scene-specific task) using the low-powered camera. It guides its wearer to turn in specific directions during an indoor navigation task. Furthermore, ensuring that the user is aware of the system’s power drain is important, and we profiled Navigator (details in Section 5) to ensure users understand the battery life in various settings. The key contributions of this work are:

- We developed Navigator to address the aforementioned challenges. Navigator is a low-cost, low-power, microcontroller-based system that runs TinyML for real-time arrow detection. It uses a ToF sensor along with a low-powered camera to provide scene-specific navigational details. In this paper, we present the design and development steps of a microcontroller-based indoor movement assisting system.
- We performed a feasibility study with 10 participants and identified the optimal parameters for the on-device deep learning. We used the learning and performed a semi-controlled study with 10 new participants. Overall, Navigator could detect arrows with a TPR of 95% and no false positives.
- We compare the power draw from various working modes of Navigator and observed that when Navigator continuously transfers data over Wi-Fi, the power draw is 419.47 mW, which reduces to 268.17 mW when most the image processing is done on device and data transfer only happens occasionally. This results in over 36% reduction in power draw.

Overall, we believe that Navigator will allow the development of several microcontroller-based real-time systems that will be capable of performing several real-time tasks.

2 RELATED WORK

Enabling assistance-free navigation for BVI individuals has been of interest in accessibility research for several years. Researchers have worked towards developing various systems to prevent users from colliding with obstacles [14] or have analysed different amounts of control for individuals during the navigation task [23]. Several proposed navigational assistant systems rely on sensors, such as ultrasonic sensors [7, 15]. Ultrasonic sensors, however, have high latency and are susceptible to interference. In comparison, ToF sensors provide more accurate range details.

Researchers have proposed several form factors for these navigational assistance devices, such as walking canes [23] and smartglasses [25]. As our focus is on eyeglasses, we discuss various eyeglasses (and wearable cameras) for navigational purposes. Zhang et al. used the accelerometer for navigational purposes [26], while Maruyama et al. developed a smartglass that provided augmented reality [19]. In terms of wearable cameras, Mustafa et al. used a 5-megapixel camera along with a Raspberry Pi for navigational assistance [20]. Several such eyeglasses used multi-sensor fusion to assist BVI individuals in navigating indoor spaces [10, 13]. More recently, Sathya et al. developed an ESP-CAM-based navigational assistance system that uses an RPi Zero and other components [25].

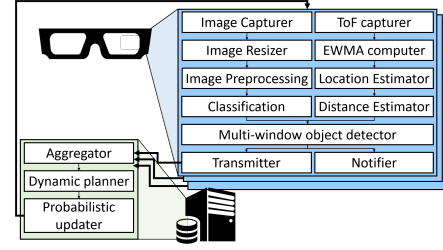


Figure 2: High level architecture of Navigator.

In all these systems, either the power consumption was high, or the system did not perform deep learning on the device itself, leading to latency and privacy concerns. Indeed, while using camera sensors to detect details in a scene, concerns about privacy in captured images are a challenge. Users are primarily concerned about who will view the images [8]. Unlike these systems, Navigator performs the data processing in situ, thus providing privacy-preserving real-time navigational assistance.

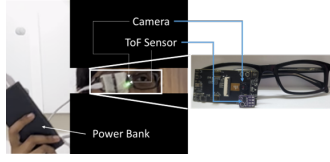
TinyML is a subfield of machine learning that aims to enable machine learning on low-powered and low-resourced embedded systems [22]. TinyML provides several libraries to reduce the regular model size so that it can fit in the memory of an embedded system. Recent work, such as that by Alharbi et al., has deployed deep learning models on embedded systems for human activity recognition [9]. Similarly, Lee et al. performed on-device machine learning for intermittent powered systems [16]. Our work is similar to these works in terms of on-device model deployment. However, none of these works focused on using images from a low-powered camera for detecting arrows or power-profiling the device.

3 ARCHITECTURE AND FEASIBILITY STUDIES

In this section, we outline the architecture of our proposed system, and then delve into the implementation details. Figure 2 presents a high level architecture of Navigator. Navigator comprises of two distinct components, namely the smartglass and the server:

Smartglass: The smartglass has the following four responsibilities: (i) capturing images and processing them on device to detect objects, (ii) capturing the ToF information to determine distance to the nearest obstacle, (iii) notifying the user about the obstacle, if necessary, and (iv) (optional) transmitting the object information and metadata to a central coordinator. Specifically, the smartglass’ image capturing module opportunistically captures images from a low power camera, resizes those images so that it can be passed onto a neural network, and performs image processing on device using TFLite micro libraries. Once the image is obtained, specific preprocessing steps are performed to remove images with low information. Images with information are next passed through a classifier that executes on the ESP32-S3 microcontroller. In parallel, the ToF sensor captures distance information. The ToF sensor’s data is passed through filters, which smooths it using prior data to estimate the distance from the object. Finally, information from the two sensors are fused and the end user is notified about the object via audio feedback. Information about obstacles is transmitted to the server on a need basis.

Server: The server component in Navigator is an optional component. The server aggregates information from multiple smartglasses



(a) Prototype of Navigator used in the user study. The left image shows a participant wearing the device during the user study. The right image shows the break out boards used for developing Navigator.



(b) Images captured by Navigator during user study. Left images contain arrows, while there are no arrows in the right images. Images are of size 96×96 .

Figure 3: The prototype of Navigator used in the user studies, along with the captured images from the camera

in vicinity. It uses this information for dynamic route planning for various smartglass users. Additionally, it can use the data from the different smartglasses for collaborative sending of the environment.

In this paper, we focus on: (a) Automatic detection of objects on the smartglass, (b) Provide feedback to the user in real time, and (c) Analyze power requirements of running the system.

3.1 Navigator Implementation

Figure 3a presents the current prototype of Navigator. The Navigator eyeglass uses the ESP32S3EYE breakout board that consists of an embedded ESP32-S3 microcontroller with 8 MB Octal PSRAM and an 8 MB flash memory. It also has an embedded RF module and a dual-core MCU. An OV2640 camera [4] with a maximum resolution of 1600×1200 is connected to the ESP32-S3. The camera is placed on the right eye of the glass. We attached a Time-of-Flight sensor (VL53L0X [6]) to the smartglass. The ToF sensor is placed just below the RGB camera module, and connected by jumper cables. In the current implementation, the ESP32-S3 is powered by a power bank attached via its serial port. However, it can be powered by a 3.7V LiPo battery with a 5V miniboost converter.

Although the camera is capable of capturing images of 1600×1200 , however, we used grayscale images of size 96×96 to develop small-sized models that would fit into the microcontroller and help reduce power consumption. Figure 3b presents the images captured by Navigator during the user study. The images from the camera were used for the following purposes – detecting whether an arrow was present in the field of view of the camera, and the arrow’s direction.

Once the image is processed, Navigator provides APIs to publish the images, distance from obstacle, presence of arrow (bool), and the classification outcome with confidence to the server or other authorized apps. Currently, we have developed a pub-sub model, where the information is present in a queue in the microcontroller. The server polls the queue to obtain the latest information. To ensure power saving, if the polling is too frequent, the smartglass responded back with a ‘No Change’ flag. Currently, on the server, we have implemented the aggregator module. In the future, we will implement the dynamic planner and the probabilistic updater modules.

3.2 On-Device Detection & Classification

We implemented two models – (i) a Faster Objects, More Objects (FOMO) model for detecting arrows in the Field of View (FoV), and (ii) a classification model for detecting the arrow’s direction. The FOMO model breaks the image into 8×8 segments and returns the probability of the presence of an arrow in each of these segments. The arrow might be distributed across multiple 8×8 pixels as the user walks towards the arrow. When the user is close to the signage

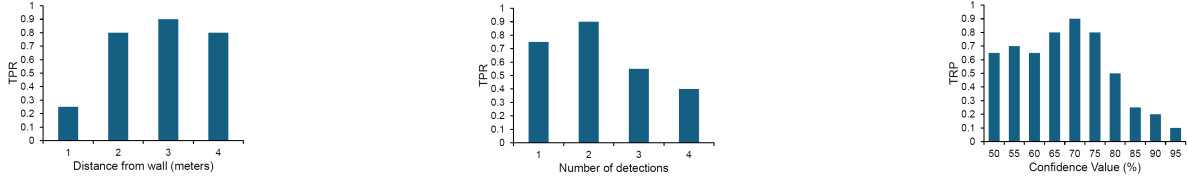
(determined by the ToF sensor), the number of pixels covered is increased gradually until the FOMO outputs whether the arrow is in the left half of the frame or the right. The classification model then starts and uses the image frame to determine the pointing direction.

Model training: To train the model, we collected approximately 400 grayscale images of different arrows using the OV2640 camera. The arrow images were of various shapes and sizes, and were collected in various environments and lighting conditions. We augmented the dataset with images obtained by rotating and changing the brightness and saturation of the original images. We used the dataset to train a 96×96 grayscale image sized MobileNetV2 model [24], and an Edge Impulse-based FOMO model [2]. We performed the quantization step to reduce the size of the model. The final size of the FOMO model was 81 KB, and the classification model was 610 KB, small enough to fit them in the microcontroller. Although the ESP32-S3 has an 8MB PSRAM, by default, the models are stored in the DRAM. We set the `MALLOC_CAP_8BIT` and `MALLOC_CAP_SPIRAM` flags in the TFLite micro library to ensure that the PSRAM was properly utilized.

Arrow detection and classification logic: We used a score-based ranking technique to detect arrows. The score starts from 0, and is incremented by 1 for a right arrow, and decremented by 1 for a left arrow, and remains unchanged if no arrow is detected. We set a *Max-arrow-threshold* ($Thres_{max}$); if the score reaches $1 \times Thres_{max}$, we declare that a right arrow is detected, and the device will notify the user to turn right; if the score reaches $-1 \times Thres_{max}$, we declare that a left arrow is detected, and the device will notify the user to turn left. However, to ensure that the arrow detection happens close to the arrow signage, we use the ToF sensor. The ToF sensor detects the distance of the user from the signage. If the user is more than a certain distance ($d_1 = 2$ meters), and the score value crosses $\pm Thres_{max}/2$, then the score is not changed, i.e., maintained at $Thres_{max}/2$. Navigator ignores readings from the ToF sensor when the user is within the specified distance of the signage, i.e., $\leq d_1$ from the signage. Furthermore, if the user is in danger of colliding with the signage, i.e., $\leq d_2 = 0.8$ meters, then the user is notified to stop and step back. This prevents the user from colliding with the signage.

3.3 Feasibility Study and Parameter Choice

We performed a user study with 10 participants (6 males, 4 females) in an IRB approved user study. Participants visited the laboratory, where they were instructed to walk in a pre-defined path whose straight line distance varied from 5 to 15 meters. The research team did not correct participants in case they deviated slightly from the path. We removed all physical obstacles from this path and placed an arrow printed on a paper approximately 1.75 meters above the ground. The Navigator device was expected to detect the arrow, and



(a) Variation of TPR as the distance from the wall changes

(b) Variation of TRP for various smoothing factor values.

(c) Variation of TRP for various values of detection confidence thresholds.

Figure 4: Choice of parameters for implementation of Navigator.

identify its pointing direction. Through this study, we attained two objectives – determine the system level parameters for running a free-living study, and ensure that the models could execute on the microcontroller. In this study, the system did not provide automatic feedback, but rather a member of the research team instructed the participant to turn. Through this study, we decided the distance threshold, smoothing factor, and detection confidence.

Distance threshold: Our goal was to ensure that the Navigator system could detect arrows from a reasonable distance. Providing the voice command too early would result in the person turning earlier than expected, while providing the command too late would result in the participant colliding with the arrow. This parameter allowed the optimal distance for prediction announcement that provided the best TPR, while allowing participant to turn correctly.

Smoothing factor: If we considered only a single instance of the machine learning model’s output, it could lead to providing a false feedback to the user. To overcome this, we considered providing an output based on the smoothed outcome of a window of predictions. Expecting multiple true outcomes to estimate the direction of arrows might lead to false negatives. However, if we consider too few outcomes to identify the direction of arrows, we might have several false positives. This factor thus allowed determining the window size over which the majority voting should be performed.

Detection confidence: Finally, a machine learning model detects arrows with a specific confidence. In case one expects to detect an arrow only when the confidence of the machine learning model is high, it is possible to miss out on several arrows. However, using the model’s outcome even when the confidence of the model is low leads to high false positives. This parameter allows balancing TPR.

We experimented with various values of distance threshold, smoothing factor, and detection confidence. We used 75% as the default value of detection confidence, 2 as the default value of distance threshold, and 3 as the default value of smoothing factor. Overall, we observed that varying one among these three parameters at a time, we obtained the best performance for Distance threshold of 3 meters, Smoothing factor of 2 detection, and detection confidence of 70%. Figure 4 presents the variation of performance for each of these parameters.

4 REAL-TIME NAVIGATION

We performed a free living study with 10 new participants (6 males, 4 females) in our laboratory. Participants were asked to walk from a pre-defined starting point in a newly chosen obstacle course. The participants were instructed to navigate based on instructions provided by Navigator. Members of the research team were instructed not to communicate with the participants, except in scenarios where participants could injure themselves. None of the data collection

process required any intervention from the research team, however. In the new obstacle course, there were 4 arrows (2 left, 2 right) in the entire path that the Navigator system was expected to detect.

4.1 Parameter settings

Using the feasibility study outcome, we set the following parameters for the study– The minimum distance threshold as 2 meters. Although 3 meters provided the highest performance, yet we chose 2 meters due to the limitation of the ToF sensor used in Navigator. The minimum number of arrows that the algorithm detected in a particular direction to provide feedback to turn in a particular direction was 2. Next, every arrow was detected with a particular detection confidence. Overall, we identified that the model provided the balance between false positives and false negatives at 70% confidence, which we used as the confidence parameter in the study.

4.2 Results

With the 10 participants in the user study, we expected overall 40 arrows to be detected in the user study. Overall, we observed that in 32 (80% of) cases, the arrow was detected while the person was walking towards the arrow. In all other cases, the ToF sensor detected that the person was within 0.8 m from the obstacle, and yet the arrow was not detected. In such cases, the system requested the participants to take a step back and move their head around slowly. Out of all such cases, in 6 cases, the system was able to detect the correct arrow with an average latency of 6.5 seconds (St. Dev. of 3 seconds) after the model requested the participant to step back. In the other two instances, based on the participant’s location (obtained by a separate indoor positioning system, not described in this paper), the default direction of turning for that location was provided to the participant after a timeout of 15 seconds. We considered both these misses as true negatives. We plotted the arrow detection latency across episodes, and observed that 80% arrow detection happens as the participant walked towards an arrow (latency: 0 second), and 95% instances were correctly detected within the latency threshold of 15 seconds.

We further analysed the data for the two false negative cases, where, even after a duration of 15 seconds, no arrow was detected. On a closer look, we observed that the detection confidence was low as the arrow was not within the field of view while the participant was walking towards the arrow. Even in such cases, Navigator ensured that participants did not collide with the obstacle.

5 POWER EVALUATION

In this section, we estimate the power consumption of the Navigator during operation. Intuitively, reducing the sensing frequency reduces the power consumption, but that increases inference latency and

reduces accuracy. Thus, apps running on energy constrained devices must reasonably trade-off between accuracy, energy and latency [11]. Although Navigator’s current prototype performs the sensing task at the highest frequency, we anticipate that introducing duty cycling approaches will reduce the power consumption. We model the power consumption of the Navigator, with that addition, but must first measure the consumption of the current prototype. We used a Monsoon Power Monitor (Monsoon Solutions Inc., FTA22J [3]) to conduct all power measurements. For each measurement, we used the Monsoon to recorded power data for half a hour at each activity level, from which we calculated the average power consumption.

We first define three different modes of operation for Navigator: (a) verbose mode (V_{ver}), (b) development mode (V_{dev}), and (c) deployment mode (V_{depl}). In V_{ver} , the camera captures images at 1 fps and performs image recognition on each image. Although the camera used can capture at a faster sampling rate, due to the image processing latency involved, we set the sampling rate of the camera to 1 HZ. The ToF sensor captures distance at 1 Hz. This ensures that every captured image has a corresponding ToF reading. Navigator is continuously connected to the server via a Wi-Fi connection. The following items are transferred to the server: (i) all images captured by Navigator, (ii) all predictions made for each image, (iii) all distance readings captured by the ToF sensor. In V_{dev} , the camera captures images at 0.33 fps and performs image recognition on each image. In case an arrow is detected, the frame rate is increased to 1 fps. We currently evaluated with 0.33 fps as the average walking speed of BVI individuals is 432 meters in 10 minutes [17], which is equivalent to a speed of approx. 2 meters in 3 seconds; 2 meters is the threshold we considered for the ToF sensor. We could consider a higher fps, if necessary. The Wi-Fi is turned on at that moment and the following items are transferred to the smartphone: (i) last 3 images captured by Navigator when a state change happened (i.e., an arrow was detected), (ii) when a prediction change happens, i.e., if an arrow is detected or stopped detecting, (iii) distance readings captured by the ToF sensor. In V_{depl} , the camera captures at 0.33 fps and performs image recognition on each image. In case an arrow is detected, the frame rate is increased to 1 fps. The Wi-Fi is turned on once per minute during the periods when arrow is detected. The only information sent to the server are the arrow direction when an arrow is detected, along with the distance to the arrow. However, this is done once every minute, thus resulting in negligible power draw. We model the power consumption of the V_{ver} , V_{dev} , and V_{depl} modes of Navigator as follows:

$$V_{ver} = S \times P_b + (1 - S) \times (P_b + P_w + P_{s1} + P_{p1} + P_d)$$

$$V_{dev} = S \times P_b + T \times (P_b + P_{s2} + P_{p2}) + (1 - (S + T)) \times (P_b + P_w + P_{s1} + P_{p1} + P_d)$$

$$V_{depl} = S \times P_b + T \times (P_b + P_{s2} + P_{p2}) + (1 - (S + T)) \times (P_b + P_{s1} + P_{p1}) + (1 - (S + T)) \times V \times (P_w + P_d)$$

Here S is the fraction of time spent in the baseline mode (this can also be modeled in terms of deep sleep), and P_b indicates the power consumption in the baseline mode – where no code exists in the setup() and loop() functions. We observed that the power consumption in P_b was 257.06 mW.

Table 1: Component wise power consumption at default setting of 1 fps image capturing, 1 distance reading per second, and 1 frame transfer over Wi-Fi per second.

	Avg. Power (mW)
Baseline (P_b)	257.06 mW
Wi-Fi On (P_w)	+291.72 mW
Camera and ToF On (P_{s1})	+2.04 mW
Camera and ToF On (0.33 Hz) (P_{s2})	+0.96 mW
On-device processing (P_{p1})	+17.74 mW
On-device processing (0.33 Hz) (P_{p2})	+9.69 mW
Data transfer (P_d)	+13.31 mW

Next, we turned on the Wi-Fi initialization and connected it to a particular access point. In this step, no data transfer happened between the ESP32-S3 and the AP. The turning on of Wi-Fi led to an increase in power draw of $P_w = 291.72$ mW. P_{s1} indicated the power draw when the camera and ToF sensor sample at 1 Hz. When sampling at 1 Hz, the power draw is 2.04 mW. When the sampling rate was reduced to 0.33 Hz (P_{s2}), the power draw reduced to 0.96 mW.

We performed on device image processing. At 0.33 Hz, the power draw was 9.69 mW. This increased to 17.74 mW when the sampling rate (and data processing rate) was increased to 1 Hz, and 50.73 mW when the sampling and processing rate was 5 Hz. Overall, we set the default value of sampling and processing rate to 1 Hz, and thus the value of $P_p = 17.74$ mW. For transferring the processed data and images, the total power consumption was $P_d = 13.31$ mW.

If we assume that the user is moving 50% of the time, and is in a state of rest during other times, we can keep the device in P_b for 50% cases, i.e., S can be set to 50%. This indicated that the device’s sensors are in its state of sleep half the time. The value of S will change based on the time of day and the individual’s lifestyle. Periodic wake-up and scans will allow Navigator to determine if it has moved out of the P_b state. Next, for the remaining 50% cases, arrow detection is a rare event. If we assume that arrows will not be detected in more than 20% cases, $T = 0.5 \times 0.8 = 0.4$. In real world, the number of times we encounter arrows is low, and we believe 20% is an optimistic upper limit. Finally in the remaining $(1 - (0.5 + 0.4) = 0.1)$ instances, we might need to turn on the Wi-Fi to transfer data once a minute. If Wi-Fi is in use for 10 seconds, then the value of V will be 0.16. Thus, the power consumption in the three states, V_{ver} , V_{dev} , and V_{depl} will be 419.47 mW, 293.8 mW, and 268.17 mW, indicating a reduction of 36.07% power draw when using V_{depl} , as compared to V_{ver} . Assuming using a 1000 mAh battery, Navigator can operate for over 12 hours in the V_{depl} mode. The value will depend on usage.

6 DISCUSSION AND FUTURE WORK

Although Navigator is capable of assisting in arrow detection and collision prevention, there are several improvements required to make it into a deployable system, which we discuss next.

Detecting beyond arrows: Although we tested the detection of only two pairs of arrows in the current user studies, Navigator can detect a diverse set of arrows. Indeed, the dataset used to train the model did not comprise of the four tested arrows. Additionally, Navigator is capable of detecting beyond arrows. In a limited study, we tested the possibility of detecting five everyday objects in a controlled lab environment using Navigator. Navigator could detect these objects with accuracy substantially better than a random guess using an

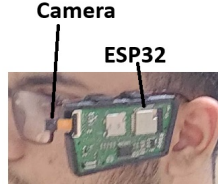


Figure 5: Evolution of Navigator: Custom printed PCB that attaches to the side of a regular glass handle.

on-device model. However, further experimentation and user studies are necessary to determine Navigator’s performance in a multi-class detection task. As a next step, we will incorporate the object detection capability in Navigator and using the same feedback system, we will provide an improvised navigation system.

Form Factor: Usability and comfort are important deciding factors in wearables. The implementation of Navigator presented in this paper does not focus extensively on the usability aspects. The prototype used off-the-shelf breakout boards for its implementation, which makes the device bulky. Furthermore, attaching a power bank affects usability. We have recently designed an updated smartglass with a custom PCB that uses the same components as the breakout boards. The microcontroller and associated circuitry are embedded on the side handle of the glass, while the camera is placed facing forward, as seen in Figure 5. This prototype still uses a power bank. In subsequent prototypes, we will attach a small battery to the other glass handle to make the system more usable.

Orthogonally, one might wonder if an eyeglass-based wearable is the correct form factor for Navigator. Indeed, several BVI individuals do not wear eyeglasses. We believe attaching the system to the glasses’ handle suits people accustomed to wearing eyeglasses. For others, we can consider developing a chest-worn form factor, like the one proposed by Mamish et al. [18]. We have currently experimented with the eyeglass form factor. However, in the future, we will experiment with other form factors to determine acceptability.

On-device sensors and power consumption: Our initial prototype of Navigator uses a camera and ToF sensor that are sampled continuously. In future, we can consider a hierarchical turning on approach, where a low power consuming sensor will turn on the more expensive sensor. This will require novel heuristic-based approaches. In terms of the ToF sensor, Navigator currently uses a single pixel ToF sensor. In future, we plan to use a ToF sensor array with 8×8 ToF sensors attached in a matrix form for improving detection accuracy.

Collaborative navigation: A collaborative navigation assistance system might help in reducing power consumption, and increase the field of view for all individuals. Obstacles in a user’s blind spot might be visible in another user’s field of view. However, the system design must be aware of privacy concerns. In future, we plan to develop a collaborative navigation system in which each device will perform image processing locally, and will share information with other devices in the proximity through the server module.

7 CONCLUSION

In this paper, we present the design and deployment of Navigator, a smartglass-based system with multiple sensors that can assist in indoor navigation. We present the design choices of Navigator in

this paper. Via a feasibility study, we demonstrate that Navigator can detect arrows with a TRP of 95%. Furthermore, we demonstrate that Navigator can be used for over 12 hours using a small sized battery.

ACKNOWLEDGMENT

Sougata Sen’s work is partially supported by a gift from the TensorFlow team to develop educational resources, and partially by Department of Science & Technology, Government of India’s grant number DST/INT/USA/NSF-DST/Sougata/P-12/2024 (G). Surjya Ghosh’s work is partially supported by AI4CPS Innovation Hub (IIT Kharagpur), and partially by the SURE grant (SUR/2022/001965) of Science and Engineering Research Board of the Department of Science & Technology, Government of India. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of the funding organizations.

REFERENCES

- [1] Esp32-s3. <https://www.espressif.com/en/products/socs/esp32-s3>, 2025.
- [2] Fomo: Object detection for constrained devices. <https://edge-impulse.gitbook.io/docs/>, 2025. Accessed:04/18/2025.
- [3] Monsoon high voltage power monitor. <https://www.monsoon.com/high-voltage-power-monitor>, 2025. Accessed:04/18/2025.
- [4] Ov2640. <https://www.arducam.com/ov2640/>, 2025. Accessed:04/18/2025.
- [5] Tensorflow lite for microcontrollers. <https://www.tensorflow.org/lite/microcontrollers>, 2025. Accessed:04/18/2025.
- [6] VL5310x tof ranging sensor. <https://www.st.com/en/imaging-and-photonics-solutions/vl5310x.html>, 2025. Accessed:04/18/2025.
- [7] ABUSUKHON, A. IOT Bracelets for Guiding Blind People in an Indoor Environment. *Journal of Communications Software and Systems* 19, 2 (2023), 114–125.
- [8] AHMED, T., HOYLE, R., CONNELLY, K., CRANDALL, D., AND KAPADIA, A. Privacy concerns and behaviors of people with visual impairments. In *Proceedings of the Annual ACM Conference on Human Factors in Computing Systems* (2015), CHI ’15.
- [9] ALHARBI, R., ET AL. Smokemon: Unobtrusive extraction of smoking topography using wearable energy-efficient thermal. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 6, 4 (2023).
- [10] BAI, J., ET AL. Smart guiding glasses for visually impaired people in indoor environment. *IEEE Transactions on Consumer Electronics* 63, 3 (2017), 258–266.
- [11] CHU, D., ET AL. Balancing energy, latency and accuracy for mobile sensor data classification. In *Conference on Embedded Networked Sensor Systems* (2011), SenSys.
- [12] DAVID, R., ET AL. Tensorflow lite micro: Embedded machine learning for tinyml systems. *Proceedings of Machine Learning and Systems* 3 (2021).
- [13] DONG, E. ighlass: A glasses-based rsrch platform for future wearable technologies.
- [14] KATZSCHMANN, R. K., ARAKI, B., AND RUS, D. Safe local navigation for visually impaired users with time-of-flight and haptic feedback device. *IEEE Trans. on Neural Systems & Rehab. Engineering* 26 (2018).
- [15] KUSNENIWAR, H. G., GHOSH, S., AND SEN, S. Soniglass: An obstacle detection and navigation system using smartglass-based ultrasonic sensors. In *International Conference on COMmunication Systems & NETworks (COMSNETS)* (2024), IEEE.
- [16] LEE, S., ET AL. Intermittent learning: On-device machine learning on intermittently powered system. *ACM on Inter., Mob., Wearable and Ubiqu. Tech.* 3, 4 (2019), 1–30.
- [17] LIU, X., ZHANG, S., ZENG, J., AND FAN, F. Analysis and optimization strategy of travel system for urban visually impaired people. *Sustainability* 11, 6 (2019), 1735.
- [18] MAMISH, J., ET AL. Nir-sighted: A programmable streaming architecture for low-energy human-centric vision applications. *ACM Transactions on Embedded Computing Systems* 23, 6 (2024), 1–26.
- [19] MARUYAMA, K., ET AL. Smart glasses for neurosurgical navigation by augmented reality. *Operative neurosurgery* 15, 5 (2018), 551–556.
- [20] MUSTAFA, A., ET AL. Intelligent glasses for visually impaired people. In *Conference on Computational Intelligence and Communication Networks (CICN)* (2022).
- [21] PESUDOV, K., ET AL. Global estimates on the number of people blind or visually impaired by cataract. *Eye* (2024), 1–15.
- [22] RAJAPAKSE, V., ET AL. Intelligence at the extreme edge: A survey on reformable tinyml. *ACM Comput. Surv.* 55, 13s (jul 2023).
- [23] RANGANENI, V., ET AL. Exploring levels of control for a navigation assistant for blind travelers. In *Conference on Human-Robot Interaction* (2023), HRI.
- [24] SANDLER, M., HOWARD, A., ZHU, M., ZHMOGINOV, A., AND CHEN, L.-C. Mobilenetv2: Inverted residuals and linear bottlenecks, 2019.
- [25] SATHYA, V., ET AL. Smart navigation glasses for independent living. In *International Conference on Computing and Communication Technologies (ICCT)* (2025).
- [26] ZHANG, Y., HU, W., XU, W., WEN, H., AND CHOU, C. T. Naviglass: Indoor localisation using smart glasses. In *EWSN* (2016), pp. 205–216.